

Cours de Programmation en C

Travaux Pratiques

— TP6 —

Exercice 1 – Fonction `strcmp`

On cherche à ré-écrire le code de la fonction `strcmp` qui compare les chaînes `s1` et `s2` suivant l'ordre lexicographique¹. Le résultat est négatif si `s1 < s2`, 0 si `s1 == s2` et positif si `s1 > s2`.

Le prototype de la fonction est `int strcmp( char* s1, char* s2)`.

Dans un même fichier `strcmp.c` :

1. Écrire une fonction `strcmp1` qui réalise la comparaison de deux chaînes en utilisant l'opérateur d'indexation `[ ]`.
2. Écrire une fonction `strcmp2` qui réalise aussi la comparaison, mais en utilisant exclusivement des pointeurs que l'on incrémente.
3. Écrire un programme principal qui saisit deux chaînes, puis affiche le résultat de la comparaison, en utilisant les fonctions `strcmp1` et `strcmp2` et la vraie fonction `strcmp`.

Rappel : il est possible de comparer deux caractères (`char`) de la même manière que l'on compare des entiers (si `a` et `b` sont des variables de type `char`, alors le code `a < b` renvoie 1 si `a` est plus petit (dans le sens alphabétique) que `b`, et 0 sinon). De plus, on n'utilisera pas de fonctions de la bibliothèque standard `string` pour écrire `strcmp1` et `strcmp2`.

Exercice 2 – Fonctions de recherche

1. Écrire la fonction `strchr` qui cherche un caractère dans une chaîne de caractères, et renvoie un pointeur vers la 1ère occurrence de ce caractère.
2. Écrire la fonction `strstr` qui cherche la 1ère occurrence d'une chaîne de caractères `target` dans une chaîne `str` (et renvoie donc un pointeur sur cette 1ère occurrence) : `char* strstr( char* str, char* target)`.

Exercice 3 – Fonction `upper`

On veut écrire une fonction `upper` qui transforme une chaîne de caractères en la mettant en majuscule. Cette fonction transformera sur place la chaîne (`char*`) passée en argument. Seuls les pointeurs seront utilisés.

Exercice 4 – Verlan

1. Écrire une procédure qui prend en paramètre une chaîne de caractères (`char*`) et la transforme en *verlan* : tous les mots de cette chaîne sont inversés (la dernière lettre devient la première), mais l'ordre des mots reste identique. Par exemple, la chaîne `"winter_is_coming."` sera changée en `"retniw_si_gnimoc."`.

Les espaces ou tabulations séparent les mots (il peut y en avoir plusieurs entre chaque mot). Les caractères de ponctuation ne sont modifiés.

¹L'ordre lexicographique est l'ordre utilisé dans les dictionnaires. Il s'agit de l'ordre alphabétique pour la 1^{ère} lettre, puis si elles sont égales, de l'ordre alphabétique pour la 2^{ème}, puis pour la 3^{ème}, etc.

2. Tester la fonction avec quelques exemples

"Il_pleut,_il_pleut_bergère" ⇒ "lI_tuelp,_li_tuelp_erègreb"

"Un_code_non_documenté_est_un_code_bon_à_jeter" ⇒ "nU_edoc_non_étnemucod_tse_nu_edoc_nob_à_retej"

3. Modifier le programme pour qu'il convertisse en verlan les phrases passées en arguments sur la ligne de commande qui appelle votre programme (Ex: ./verlan.out "il faut beau" "il fait chaud" "ceci est un test"). Pour cela, la fonction principale main peut prendre deux arguments, traditionnellement appelé argc (un entier) et argv (un tableau de chaînes de caractères) :

```
int main(int argc, char *argv[])
```

Le programme devra transformer en verlan et afficher chacun des phrases passées en arguments (avec des guillemets autour des phrases, pour que le shell ne découpe pas mot à mot votre phrase). Dans /home/sasl/shared/ei-ii/C/TP6/ se trouvera le fichier jules.txt contenant des phrases (déjà entourées de guillemets), et vous pourrez les utiliser en argument pour tester votre programme (./verlan.out `cat jules.txt`).